第三阶段、DeepAgents 核心功能介绍

注意：没有学习 LangChain 的用户，建议先学习 LangChain 1.0 的基础内容，再学习 DeepAgents。

一、安装环境与依赖

1.1 Python 环境要求

```
# Python 版本要求
python --version
# 输出: Python 3.11.14
```

1.2 安装 DeepAgents

```
# 安装 deepagents 依赖
pip install deepagents
```

1.3 安装网络搜索工具（可选）

```
# 安装 Tavily 搜索工具
pip install langchain-tavily

# 安装美化代码库 Rich（用于调试输出）
pip install rich
```

1.4 查看版本信息

```
# 查看相关包版本
pip list | grep -E 'langchain|deepagents'
```

二、核心入口：create_deep_agent()

这是整个框架的核心函数，它创建了一个功能完整的深度智能体。

源码位置：D:\sorfware_install\python3.8_install\envs\deepagent_1\Lib\site-packages\deepagents\graph.py

```python
def create_deep_agent(  # noqa: C901, PLR0912  # Complex graph assembly logic with many conditional branches
    model: str | BaseChatModel | None = None,
    tools: Sequence[BaseTool | Callable | dict[str, Any]] | None = None,
    *,
    system_prompt: str | SystemMessage | None = None,
    middleware: Sequence[AgentMiddleware] = (),
    subagents: list[SubAgent | CompiledSubAgent] | None = None,
    skills: list[str] | None = None,
    memory: list[str] | None = None,
    response_format: ResponseFormat | None = None,
    context_schema: type[Any] | None = None,
    checkpointer: Checkpointer | None = None,
    store: BaseStore | None = None,
    backend: BackendProtocol | BackendFactory | None = None,
    interrupt_on: dict[str, bool | InterruptOnConfig] | None = None,
    debug: bool = False,
    name: str | None = None,
    cache: BaseCache | None = None,
) -> CompiledStateGraph:
```

2.1 默认配置

- 默认模型：使用 Claude Sonnet 4 或 GPT-4o 作为默认模型（推荐）

```python
def get_default_model() -> ChatAnthropic:
    """Get the default model for deep agents.

    Returns:
        `ChatAnthropic` instance configured with Claude Sonnet 4
    """
    return ChatAnthropic(
        model_name="claude-sonnet-4-5-20250929",
        max_tokens=20000,
    )
```

- 集成工具：默认集成 7 个核心文件操作工具
- 待办事项：提供待办事项管理功能
- 子代理支持：支持子代理调用

2.2 关键参数

参数	说明
model	支持自定义语言模型
tools	自定义工具集
system_prompt	系统提示词
subagents	子代理配置
backend	文件存储后端
interrupt_on	人机交互配置（Human-in-the-Loop）。允许在特定节点暂停 Agent 执行，等待人工干预。

2.3 基础使用示例

```
from deepagents import create_deep_agent
from langchain_tavily import TavilySearch
from langgraph.checkpoint.memory import InMemorySaver
from dotenv import load_dotenv
from langchain_deepseek import ChatDeepSeek

# 1. 加载 .env 环境变量
load_dotenv(override=True)

# 2. 初始化模型
model = ChatDeepSeek(model="deepseek-chat", temperature=0)

# 3. 初始化 Tavily 搜索工具
tavily = TavilySearch(max_results=3)

# 4. 编写系统提示词
research_instructions = """
您是一位资深的研究人员...
"""

# 5. 创建 DeepAgents 智能体
agent = create_deep_agent(
    name="DeepAgents_Agent",
    tools=[tavily],
    model=model,
    system_prompt=research_instructions,
    checkpointer=InMemorySaver(),
)

# 6. 配置线程 ID
config = {"configurable": {"thread_id": "1"}}

# 7. 调用智能体
result = agent.invoke(
    {"messages": [{"role": "user", "content": "查询内容"}]},
```

```
    config=config
)

# 8. 获取结果
print(result["messages"][-1].content)
```

2.4 研究完成总结

- Agent 会自动完成以下工作：

三、create_deep_agent 内部结构

3.1 源码参数结构

```
create_deep_agent(
    name: str,
    model: BaseChatModel,
    tools: list[BaseTool],
    system_prompt: str,
    subagents: list[SubAgentConfig],
    backend: Backend,
    checkpointer: BaseCheckpointSaver,
    interrupt_on: list[str],
    max_turns: int,
    **kwargs
)
```

Agent 加载的工具列表

类别	工具名称	描述
用户工具	tavily_search	A search engine optimized for comprehensive, accurate, and trusted results. Usef...
文件系统工具	ls	Lists all files in the filesystem, filtering by directory. Usage: - The path pa...
	read_file	Reads a file from the filesystem. You can access any file directly by using this...
	write_file	Writes to a new file in the filesystem. Usage: - The file_path parameter must b...
	edit_file	Performs exact string replacements in files. Usage: - You must use your `Read` ...
	glob	Find files matching a glob pattern. Usage: - The glob tool finds files by match...
	grep	Search for a pattern in files. Usage: - The grep tool searches for text pattern...
	execute	Executes a given command in the sandbox environment with proper handling and sec...
系统工具	write_todos	Use this tool to create and manage a structured task list for your current work ...
	task	Launch an ephemeral subagent to handle complex, multi-step independent tasks wit...

```
统计信息
共计 10 个工具

• 用户工具: 1 个
• 文件系统工具: 7 个
• 系统工具: 2 个
```

3.2 工具分类

类别	工具数量	说明
用户工具	1+ 个	用户自定义的工具（如 Tavily 搜索）
文件系统工具	7 个	用于读写、查询、执行文件系统中的文件
系统工具	2 个	待办事项管理和子代理调用

3.3 文件系统工具详细说明

工具名称	功能描述
ls	列出目录中的所有文件，支持目录过滤
read_file	从文件系统读取文件内容
write_file	将内容写入新文件
edit_file	对文件进行精确的字符串替换
glob	查找匹配 glob 模式的文件
grep	在文件中搜索指定模式
execute	在沙箱环境中执行命令

3.4 系统工具详细说明

工具名称	功能描述
write_todos	创建和管理任务列表，用于规划复杂任务
task	启动临时子代理来处理复杂的多步骤任务

四、提示词系统

DeepAgents 内置了多个提示词组件，用于控制智能体的行为。

4.1 文件系统提示词

文件路径：deepagents/middleware/filesystem.py

- 核心功能：
- 定义文件系统操作的安全规则
- 规定文件路径的处理方式
- 提供文件操作的最佳实践指导

```
FILESYSTEM_SYSTEM_PROMPT = """## Following Conventions

- Read files before editing — understand existing content before making changes
- Mimic existing style, naming conventions, and patterns

## Tool Usage and File Reading

Follow the tool docs for the available tools. In particular, for filesystem tools, use pagination (offset/limit) when reading large files.

## Filesystem Tools `ls`, `read_file`, `write_file`, `edit_file`, `glob`, `grep`

You have access to a filesystem which you can interact with using these tools.
All file paths must start with a /.

- ls: list files in a directory (requires absolute path)
- read_file: read a file from the filesystem
- write_file: write to a file in the filesystem
- edit_file: edit a file in the filesystem
- glob: find files matching a pattern (e.g., "**/*.py")
- grep: search for text within files"""
```

4.2 执行工具提示词

文件路径：deepagents/middleware/filesystem.py

- 核心功能：
- 定义工具调用的参数规范
- 说明工具返回值的处理方式
- 提供错误处理指导

```
EXECUTION_SYSTEM_PROMPT = """## Execute Tool `execute`

You have access to an `execute` tool for running shell commands in a sandboxed environment.
Use this tool to run commands, scripts, tests, builds, and other shell operations.

- execute: run a shell command in the sandbox (returns output and exit code)"""
```

4.3 待办事项提示词

文件路径：deepagents/middleware/todo.py

- 核心功能：
- 定义任务分解的格式（Markdown）
- 说明任务状态的含义（Pending/Completed）
- 提供任务管理的最佳实践

```
WRITE_TODOS_SYSTEM_PROMPT = """## `write_todos`

You have access to the `write_todos` tool to help you manage and plan complex objectives.
Use this tool for complex objectives to ensure that you are tracking each necessary step and giving the user visibility into your progress.
This tool is very helpful for planning complex objectives, and for breaking down these larger complex objectives into smaller steps.

It is critical that you mark todos as completed as soon as you are done with a step. Do not batch up multiple steps before marking them as completed.
For simple objectives that only require a few steps, it is better to just complete the objective directly and NOT use this tool.
Writing todos takes time and tokens, use it when it is helpful for managing complex many-step problems! But not for simple few-step requests.

## Important To-Do List Usage Notes to Remember
- The `write_todos` tool should never be called multiple times in parallel.
- Don't be afraid to revise the To-Do list as you go. New information may reveal new tasks that need to be done, or old tasks that are irrelevant."""  # noqa: E501
```

4.4 子代理提示词

文件路径：deepagents/middleware/subagent.py

- 核心功能：
- 定义子代理调用协议
- 规定 task 工具的参数结构
- 说明结果回传方式
- 设置安全底线

```
DEFAULT_SUBAGENT_PROMPT = "In order to complete the objective that the user asks of you, you have access to a number of standard tools."

# State keys that are excluded when passing state to subagents and when returning
# updates from subagents.
#
# When returning updates:
# 1. The messages key is handled explicitly to ensure only the final message is included
# 2. The todos and structured_response keys are excluded as they do not have a defined reducer
#    and no clear meaning for returning them from a subagent to the main agent.
# 3. The skills_metadata and memory_contents keys are automatically excluded from subagent output
#    via PrivateStateAttr annotations on their respective state schemas. However, they must ALSO
#    be explicitly filtered from runtime.state when invoking a subagent to prevent parent state
#    from leaking to child agents (e.g., the general-purpose subagent loads its own skills via
#    SkillsMiddleware).
_EXCLUDED_STATE_KEYS = {"messages", "todos", "structured_response", "skills_metadata", "memory_contents"}
```

五、规划工具系统 (Planning System / Todo List)

5.1 核心组件

- 组件：TodoListMiddleware
- 工具名：write_todos

5.2 功能说明

Agent 在行动前先生成 Markdown 格式的待办事项列表（Todo List），并在执行过程中更新状态（完成/进行中）。

5.3 工作流程

1. 任务接收：Agent 接收复杂任务
2. 任务分解：调用 write_todos 将任务拆解为子步骤
3. 状态更新：每完成一步，更新状态为 Completed
4. 自我反思：在每一步行动前，Agent 都会看到当前的 Todo List

5.4 待办事项状态

状态	说明
Pending	待执行
in_progress	进行中
Completed	已完成

六、实践案例：深度研究智能体

6.1 案例概述

1. 接收研究主题
2. 通过互联网搜索收集信息
3. 整合分析形成结构化报告
4. 将报告保存到本地文件

6.2 完整代码

```
import json
from rich.console import Console
```

```
from rich.json import JSON
from rich.panel import Panel

console = Console()

def debug_agent(query: str, agent, save_to_file: str = None):
    console.print(Panel.fit(f"[bold cyan]查询:[/bold cyan] {query}"))
    config = {"configurable": {"thread_id": "2"}}
    for event in agent.stream(
        {"messages": [{"role": "user", "content": query}]},
        stream_mode="values", config=config
    ):
        # 处理事件...
    return final_response
```

6.3 运行结果解读

步骤 1-2：接收用户查询，进行初步分析

步骤 3+：调用 write_todos 创建待办事项 -> 使用搜索工具收集信息 -> 逐步完成各个子任务 -> 最终生成报告并保存

七、总结

7.1 DeepAgents 核心价值

- 开箱即用：集成了文件系统操作、任务规划、子代理管理等核心功能
- 可扩展性：支持自定义工具和子代理
- 人机交互：提供 interrupt_on 配置，支持关键节点的审核与干预
- 持久化：支持检查点保存与恢复

7.2 适用场景

- 深度研究任务
- 复杂任务规划
- 自动化工作流
- 企业级应用

7.3 技术栈定位

DeepAgents 作为 LangChain 生态系统中的"代理工具"：

- LangChain：框架
- LangGraph：运行时
- DeepAgents：代理工具

三者形成完整的技术栈，为 AI 应用开发提供全方位支持。

八、验证代码

请参考 stage3_verification.py 文件中的验证脚本，该脚本包含以下验证内容：

- 环境验证：检查 Python 版本和依赖包
- 模型验证：测试语言模型连接
- 工具验证：检查所有内置工具是否正常加载
- 智能体验证：运行简单的测试任务

运行验证：

```
conda activate deepagent_1
python middleware_info/stage3_verification.py
```

九、create_deep_agent 的 Skills 支持【补充说明】

DeepAgents 框架通过 create_deep_agent 函数提供强大的 Skills 功能支持，允许用户加载自定义的技能文件来扩展 Agent 的能力。

9.1 Skills 概述

Skills 是 DeepAgents 框架的一个重要特性，它允许用户：

定义自定义行为规范和指令
创建可重用的技能模板
在多个 Agent 之间共享技能配置

9.2 skills 参数说明

参数	说明
skills	可选参数，技能源路径列表（如 ["/skills/user/", "/skills/project/"]）
类型	list[str] | None
默认值	None
加载方式	使用 StateBackend 时通过 invoke(files={...}) 传递；使用 FilesystemBackend 时从磁盘加载

9.3 使用示例

基本用法：

```
from deepagents.graph import create_deep_agent

# 创建带有 skills 的 deep agent
```

```
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-5-20250929",
    skills=["/skills/user/", "/skills/project/"],  # 指定 skill 路径
    tools=[...],  # 可选：自定义工具
    subagents=[...],  # 可选：子代理配置
)

# 调用 agent
result = agent.invoke({"messages": [{"role": "user", "content": "你的任务"}]})
```

子代理也支持 Skills：

```
# 在子代理配置中也可以指定 skills
agent = create_deep_agent(
    model="anthropic:claude-sonnet-4-5-20250929",
    subagents=[
        {
            "name": "researcher",
            "description": "进行深入研究",
            "prompt": "你是一位研究员...",
            "skills": ["/skills/research/"],  # 子代理专属 skills
            "tools": [...],
        }
    ]
)
```

9.4 注意事项

路径必须使用 POSIX 风格（正斜杠 /）
路径是相对于 backend 的 root 目录
相同名称的 skill，后加载的会覆盖先加载的（last one wins）
Skills 通过 SkillsMiddleware 添加到 Agent 的中间件栈中

9.5 源码解析

在 graph.py 中，Skills 的实现逻辑如下：

```
# 1. 主 Agent 的 skills 支持（第 272-273 行）
if skills is not None:
    deepagent_middleware.append(SkillsMiddleware(backend=backend, sources=skills))

# 2. 通用子代理的 skills 支持（第 210-211 行）
if skills is not None:
    gp_middleware.append(SkillsMiddleware(backend=backend, sources=skills))

# 3. 用户自定义子代理的 skills 支持（第 250-252 行）
subagent_skills = spec.get("skills")
if subagent_skills:
    subagent_middleware.append(SkillsMiddleware(backend=backend,
sources=subagent_skills))
```